Premature Hardware Binding is the Root of all Evil! Making Serverless Performance-Portable with Lazy Hardware Binding

Serhii Ivanenko
serhii.ivanenko@tecnico.ulisboa.pt
INESC-ID, Instituto Superior Técnico, University of Lisbon
Lisbon, Portugal

Carlos Segarra
cs1620@ic.ac.uk
Imperial College London
London, United Kingdom

Rodrigo Bruno
rodrigo.bruno@tecnico.ulisboa.pt
INESC-ID, Instituto Superior Técnico, University of Lisbon
Lisbon, Portugal

Abstract

Modern clouds are characterized by increasing hardware heterogeneity. While exposing hardware complexity to developers enables performance optimizations, it also constrains deployment options and fragments the platform's resource pool. Conversely, serverless platforms abstract hardware details to increase deployment flexibility. However, this forces developers to compile their applications conservatively to ensure portability, sacrificing significant performance gains of CPU-specific instruction set extensions.

To break through this tradeoff, we introduce Mosaic, a serverless platform that decouples portable application logic from performance-critical dependencies. Rather than eagerly bundling applications into a single binary, Mosaic delays binding until the deployment target is known, dynamically injecting pre-compiled, hardware-optimized library implementations tailored to the specific machine. The proposed design offers deployment flexibility, near-optimal performance, and low developer burden. Our evaluation demonstrates that Mosaic matches the throughput of aggressively optimized CPU-bound applications, achieving up to 40% higher throughput than builds without CPU-specific optimizations. Simultaneously, its cross-architecture portability prevents cluster fragmentation, reducing 90th percentile end-to-end scheduling latencies by up to 604× under constrained resources, while its architectural deduplication reduces application storage footprints by over two orders of magnitude.

CCS Concepts: • **Computer systems organization** → **Cloud computing**; • **Software and its engineering** → **Runtime environments**; **Software performance**.

Keywords: Serverless, Cloud Computing, Hardware Heterogeneity, Performance Portability



This work is licensed under a Creative Commons Attribution 4.0 International License.

SIGOPS ATC 2026, Hong Kong, China

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2818-1/2026/11
https://doi.org/10.1145/3828523.3857117

ACM Reference Format:
Serhii Ivanenko, Carlos Segarra, and Rodrigo Bruno. 2026. Premature Hardware Binding is the Root of all Evil! Making Serverless Performance-Portable with Lazy Hardware Binding. In *The 2026 ACM SIGOPS Annual Technical Conference (SIGOPS ATC 2026), November 15–18, 2026, Hong Kong, China.* ACM, New York, NY, USA, 15 pages. https://doi.org/10.1145/3828523.3857117

1 Introduction

Serverless has transformed the way developers use the cloud. Instead of controlling where and when instances are deployed, developers now rely on cloud platforms to automatically provision and scale serverless instances. Behind the curtains, cloud providers operate large fleets of computing resources characterized by increasing hardware heterogeneity [40, 48, 52]. Moreover, the end of Moore's Law pushed hardware manufacturers to develop specialized CPU instructions (e.g., AVX, AMX, SVE), on-chip CPU accelerators (e.g., DSA, IAA, QAT), and power-efficient CPUs (such as AWS Graviton) to improve the performance and power efficiency of common data center operations such as encryption, compression, and streaming. Overall, advances in hardware specialization resulted in an increase of the number of instance types offered by AWS from just 52 in 2015 to 1,057 in 2025, a 20× complexity jump that hyperscalers exposed to users [48].

Instead of directly exposing hardware selection complexity to developers, serverless platforms offer developers a single knob, allowing developers to select which Instruction Set Architecture (ISA) to use (e.g., x86-64 or ARM64). In addition to reducing the developer burden, limiting the control over which hardware to utilize also allows platforms to freely select any available hardware, as long as it offers the selected ISA. In this paper, we refer to serverless platforms as *ISA-binding* as they only require developers to bind applications to an ISA when registering an application.

Despite the infrastructure management flexibility advantages, ISA-binding platforms inherently underutilize modern hardware extensions, resulting in sub-optimal performance. Imagine a serverless application that utilizes a compression library to decompress data before processing it. In an ISA-binding platform, the application (and its library dependencies) need to be compiled to the specific ISA and cannot

make assumptions about the specific hardware where the application will run. Even if there are CPUs available featuring advanced instructions for compression (e.g., SSE or AVX [45, 53]), developers are not guaranteed that all application instances will be deployed on such hardware, and thus must conservatively compile the application to use only the standard ISA instructions.

To address this performance inefficiency, an alternative is to utilize platforms that expose the full complexity of hardware selection to developers, such as Infrastructure-as-a-Service (IaaS) offerings. We refer to these platforms as *CPU-binding*. While this approach allows developers to leverage advanced hardware extensions to maximize performance, it creates a significant burden to manually navigate the diverse hardware landscape. Furthermore, this tight coupling between applications and specific hardware configurations inevitably leads to lower resource utilization on the platform side, as cloud providers lose the flexibility to freely schedule workloads across their heterogeneous server fleets. This hardware underutilization forced hyperscalers to resort to Spot Markets [8, 11, 14], in which cloud providers offer discounts on idle capacity. Still, Spot Markets introduce further complexity as developers need to adjust application resources according to availability in Spot Markets [30, 33].

Maximizing performance in heterogeneous fleets while hiding hardware selection complexity is fundamentally challenging. Although most popular languages (Python, Java, JavaScript) offer hardware portability, most applications still rely on native libraries to execute performance-sensitive computations (e.g., machine learning, encryption, compression, and media processing). Existing attempts to bridge this gap introduce significant trade-offs. For instance, developers can pre-build applications for all possible hardware combinations, leading to high developer burden and increased storage costs. Alternatively, systems such as XaaS [34] propose delaying application compilation until it is deployed, resulting in extended cold starts. While caching compiled artifacts can mitigate this cold-start penalty, it leads to significant storage duplication. Finally, popular libraries such as OpenSSL [18] and FFmpeg [13] manually include assembly instructions optimized for different hardware and dynamically check if certain instructions are available at run-time. This approach significantly complicates implementation and maintenance, making it not viable for most commodity software.

In this paper, we set out to rethink how serverless platforms can fully leverage available hardware without exposing its underlying complexity to developers. We begin by characterizing serverless workloads using a popular benchmark suite, SeBS [35]. Our analysis of execution time and storage requirements reveals that most serverless applications naturally decompose into two distinct components: application-specific business logic and external library dependencies. Furthermore, we observe that library code heavily dominates both the overall execution time and the total

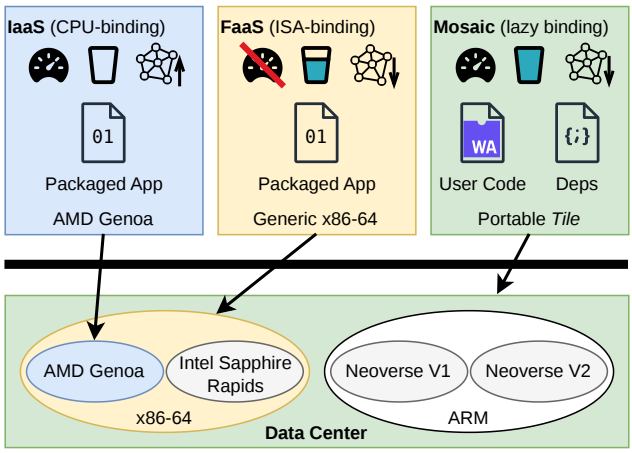


Figure 1. Cloud platform's tradeoff with heterogeneous hardware. CPU-binding enables performance optimizations (🏎) but constrains hardware utilization (🗑) and exposes developers to complex hardware selection (🧠). ISA-binding offers higher hardware utilization and lower developer effort at the expense of hardware-specific optimizations. Mosaic offers performance optimization, high utilization, and low developer effort.

package size. From this characterization, we derive a key insight: application-specific code is not performance-sensitive and can be compiled into a portable format, whereas library code is highly performance-sensitive and must be optimized for the available hardware.

Building on this insight, we propose Mosaic,[1] a novel serverless platform designed for elastic applications in heterogeneous environments. The core component of Mosaic's architecture is a *Tile*, a lazily-loaded component that can either contain portable application code or hardware-optimized library code. At deployment time, Mosaic supplies library tile implementations optimized to the hardware where the application is being deployed. Mosaic allows platforms and developers to provide specialized implementations of common performance-critical libraries. Contrary to traditional ISA-binding or CPU-binding platforms that prematurely bind application and library code to a specific ISA or CPU, Mosaic lazily binds performance-critical libraries to available hardware. This design achieves two important goals: it maximizes performance by supplying hardware-optimized code for performance-critical libraries while keeping the simplicity and deployment flexibility of serverless platforms where developers are abstracted from complex hardware selection.

To realize this lazy-binding architecture, Mosaic fundamentally changes how cloud applications are packaged (see

[1]Mosaic is open-source and available at https://github.com/cloudsys-dpss-inescid/mosaic and https://zenodo.org/records/23319839 (DOI: https://doi.org/10.5281/zenodo.23319839).

Fig. 1). Rather than eagerly linking application code alongside its dependencies into a single monolithic binary or container image, Mosaic cleanly partitions the two. Application-specific code is compiled into WebAssembly (abbreviated Wasm), a portable code format, while performance-critical libraries are compiled to the specific available hardware and are managed exclusively by the platform.

Since the platform has full observability over its hardware, it maintains a registry of popular libraries optimized for every machine type available in its fleet. When registering an application, developers simply provide a manifest detailing the required dependencies (similar to a build file). At deployment time, Mosaic loads the portable application code and dynamically links it with the corresponding library tiles, which have been pre-compiled and optimized by the platform for the specific hardware of the target machine. This architectural partition guarantees cross-architecture portability for the application logic while simultaneously ensuring that performance-sensitive dependencies execute using the most efficient instructions available on the underlying hardware. Furthermore, because dependencies are managed centrally by the platform, Mosaic naturally deduplicates common libraries across applications, drastically reducing the overall application storage footprint.

The Mosaic runtime dynamically injects the requested native libraries into the Wasm engine as host functions. However, bridging the boundary between a Wasm guest and a native host traditionally incurs prohibitive serialization overhead. To overcome this, Mosaic introduces an efficient Foreign Function Interface (FFI) trampoline that offers Wasm to native code transitions with zero-copy data passing for the most commonly used data types. This ensures that the architectural partition does not penalize the execution speed, making the lazy-binding approach fundamentally viable.

Mosaic is designed to be fully compatible with existing isolation and virtualization technologies; the runtime and its injected libraries are deployed within traditional containers or microVMs to guarantee robust multi-tenant isolation. As illustrated in Fig. 1, Mosaic can safely and efficiently schedule workloads on any machine in the data center without compromising CPU-specific optimizations or tenant security.

We evaluate Mosaic against existing eager-binding platforms using established benchmarks and real-world serverless traces. Our evaluation demonstrates that Mosaic successfully breaks the traditional trade-off between performance and deployment flexibility. Specifically, for compute-intensive workloads, Mosaic maintains execution throughput on par with aggressively optimized CPU-binding baselines, proving that our lazy-binding architecture and zero-copy trampoline impose negligible overhead (§4.3). Simultaneously, Mosaic's ability to seamlessly schedule workloads across heterogeneous architectures significantly improves fleet utilization over strict CPU-binding and ISA-binding, translating into reduced scheduling delays, lower end-to-end tail latency by 399–604×, and fewer Service-Level Agreement (SLA) breaches (§4.2). Finally, by naturally deduplicating shared libraries across tenants, Mosaic reduces the total storage footprint by more than two orders of magnitude compared to eager-binding approaches (§4.4).

In summary, Mosaic offers the following contributions:

- We characterize modern serverless workloads, revealing that applications naturally decompose into highly popular, performance-sensitive library dependencies and non-performance-sensitive, application-specific code;
- We rethink the packaging format for elastic cloud applications by decoupling the portable application logic from performance-critical dependencies;
- We demonstrate that Mosaic enables cross-architecture portability without compromising performance of serverless workloads thanks to a common portable compilation target for application logic and optimized native implementations of performance-critical dependencies, which are invoked with near-zero overhead through efficient trampolines;
- We measure the performance, cost-effectiveness, and utilization gains of Mosaic tiles by running local experiments and by deploying Mosaic in a realistic distributed scenario (§4). We demonstrate that Mosaic reduces the end-to-end tail latency by 399–604× compared to eager-binding approaches using real-world trace.

2 Background and Motivation

In this section, we detail the scale of hardware heterogeneity in modern cloud data centers and quantify the performance and cost trade-offs involved in hardware selection. Finally, we analyze the characteristics of serverless workloads to motivate our lazy-binding architecture.

2.1 Heterogeneity in the Cloud

Modern cloud data centers evolve continuously as providers adopt newer generations of server machines. A recent study on cloud hardware [48] demonstrates that this continuous evolution has established hardware heterogeneity as the standard operating environment. A typical cloud fleet consists of multiple CPU architectures (e.g., x86-64 and ARM) and a diverse mix of microarchitectures within those families (e.g., Intel Sapphire Rapids, AMD Genoa, ARM Neoverse V1). New generations and microarchitectures continuously introduce distinct hardware features. For instance, instruction set extensions such as AVX-512, VNNI, AMX in x86-64, and SVE in ARM were introduced just within the last decade. As a result, developers face a rapidly changing hardware landscape with increasingly complex hardware extensions that has grown 20× in the last decade alone [48]. Driven by the end of Moore's Law, this trend of hardware specialization is likely to continue and accelerate.

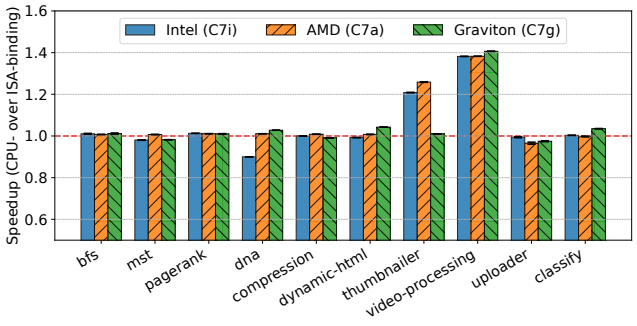


Figure 2. Latency speedup of the benchmarks compiled with CPU-binding over ISA-binding on different machines, demonstrating that CPU-specific optimizations improve performance of compute-intensive benchmarks.

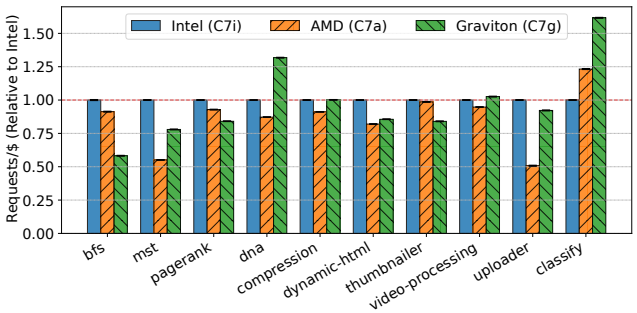


Figure 3. Cost efficiency of the benchmarks compiled with CPU-specific optimizations and executed on AWS EC2 instances [7] with Intel, AMD, and Graviton CPUs. This figure demonstrates that cost efficiency can vary across different CPU types even for the same benchmark code.

Crucially, pure performance is not the only metric that matters when choosing a suitable CPU model or Instruction Set Architecture (ISA) for running workloads in the cloud. Different architectures and machine types offer different pricing models, making cost efficiency an equally important metric for developers. Navigating the cost-performance tradeoff to find optimal machine configurations is a relevant research topic [29, 50]. For example, some machines can process up to 61.7% more requests of specific workloads for the same price than the others (see §2.2 for more details). As a consequence, the heterogeneity in the cloud results in a complex landscape where the optimal deployment target varies significantly depending on the nature of the workload and on the developer's objective to either maximize cost efficiency or pure performance.

2.2 The Eager-Binding Tradeoff

The diverse hardware landscape presents cloud platforms and developers with a fundamental tradeoff between CPU-binding and ISA-binding. Binding applications to a specific CPU maximizes performance through hardware extensions but increases developer complexity and fragments the platform's resource pool, lowering overall utilization. Conversely, elastic cloud models like Container-as-a-Service (CaaS) or Function-as-a-Service (FaaS) enforce binding to a generic ISA. While ISA-binding ensures workload portability and high resource utilization, it forces conservative compilation that leaves significant performance on the table.

Table 1. Optimal machine type and compilation profile combinations for serverless benchmarks [35], demonstrating the lack of a universal "best" configuration for either raw throughput or cost efficiency. In this table, *CPU* signifies CPU-binding compilation with CPU-specific optimizations, and *ISA* signifies ISA-binding with conservative compilation.

Benchmark	Pure Performance	Cost Efficiency
bfs	AMD - CPU	Intel - CPU
mst	Intel - ISA	Intel - ISA
pagerank	AMD - CPU	Intel - CPU
dna	Intel - ISA	ARM - CPU
compression	AMD - CPU	ARM - ISA
dynamic-html	Intel - ISA	Intel - ISA
thumbnailer	AMD - CPU	Intel - CPU
video-processing	AMD - CPU	ARM - CPU
uploader	Intel - ISA	Intel - ISA
classify	AMD - CPU	ARM - CPU

To quantify this tradeoff, we measured throughput of popular serverless benchmarks [35] compiled to generic x86 or ARM (ISA-binding) and optimized for the target CPU (CPU-binding). Fig. 2 shows that some workloads can achieve 40% higher throughput if optimized for a specific CPU. Cost efficiency is another important metric in addition to pure performance. Instances of the same size but with different machine types (i.e., different CPUs) may vary in pricing per hour and may yield different performance for the same workload. To quantify cost efficiency, we evaluated throughput per dollar (i.e., how many requests can be processed per dollar) across three equally-sized machine types in AWS EC2: `C7i` (Intel), `C7a` (AMD), and `C7g` (AWS Graviton). Fig. 3 shows that for some workloads, the increase in the number of requests per dollar can be up to 50.8–61.7% across available CPUs.

Tab. 1 summarizes performance and cost efficiency results, demonstrating the optimal combination of machine type and optimization profile (to ISA or CPU) for each benchmark in order to maximize throughput or cost efficiency. The table, along with Fig. 2 and Fig. 3, reveals three important insights:

- (**I1**) There is no single best combination of machine type and compilation profile for all benchmarks to achieve the best raw throughput or the best cost efficiency as the values are mostly different within the columns in Tab. 1;
- (**I2**) Combinations of machine type and compilation profile that prioritize performance are not the same as the ones

prioritizing cost efficiency for the same benchmarks as the values are mostly different within the rows in Tab. 1;
- (**I3**) Some benchmarks are even penalized by CPU-specific optimizations due to the overheads associated with the use of vectorization (more details in §4.3).

In summary, Tab. 1 demonstrates that hardware selection can significantly impact both performance and cost efficiency. However, finding the optimal deployment configuration is inherently complex. There is no universally best machine type or compilation profile; the ideal choice depends heavily on specific hardware capabilities, the developer's target metric, and individual workload characteristics (such as the potential overheads associated with vectorization [52]).

To bypass this tradeoff between hardware selection and complexity, some popular libraries, such as OpenSSL [18] or FFmpeg [13], use runtime capability probing (e.g., `CPUID`) to dynamically dispatch execution to multiple pre-compiled optimized paths bundled within a single binary. Alternatively, developers can use *uber-packaging* to bundle entirely separate binaries for different architectures into a single container, or delay compilation [34] to deployment time. However, these approaches inflate storage with unused assembly code, increase maintenance burden, and introduce prohibitive cold-start latencies (see §4). Furthermore, relying on developers to optimize applications for specific targets delays the adoption of new hardware. For instance, until recently, the default distribution of TensorFlow [27] did not include advanced vector extensions (AVX, SSE), forcing users to manually recompile TensorFlow from source to utilize these instructions [21]. Similarly, migrating applications to new architectures requires significant developer effort [9].

Both ISA-binding and CPU-binding impose inefficiencies for the cloud provider. Because existing packaging formats, such as container images, force an early commitment to a particular architecture or CPU, cloud applications become non-fungible across the diverse hardware. For example, a platform cannot schedule an x86-64 application container on an idle ARM-based machine. This lack of cross-architecture workload portability forces providers to operate with isolated resource pools, leading to resource underutilization.

Furthermore, even within a single architecture, eager binding complicates resource management. Scheduling CPU-bound tasks across various generations of the same CPU family can pose challenges, primarily due to the differences in CPU features, such as microarchitectural extensions. This challenge was highlighted in a recent study [40] in the context of virtual machine live-migration across different processors sharing the same architecture. To mitigate the financial impact of unused compute capacity caused by eager-binding packaging formats, cloud providers resort to economic workarounds, such as offering discounts on unused instances. Spot instances [8] in AWS EC2 [7] is one way to implement these discounts.

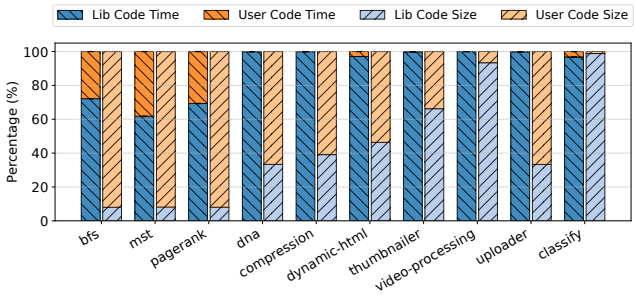


Figure 4. Relative breakdown of execution time (left bars) and binary size (right bars) of user code and library code for the SeBS benchmarks [35]. The figure demonstrates that external libraries dominate the total execution time for cloud applications and take up a significant portion of their storage footprint on average.

2.3 The Nature of Cloud Workloads

Serverless workloads commonly offload core functionality to external dependencies, especially for compute-intensive tasks such as machine learning inference, media processing, cryptography, and scientific computing. Cloud platforms offer solutions for applications relying on external dependencies, for example, AWS Lambda layers [1] allow developers to accompany their serverless functions with libraries or binaries. Furthermore, our study of typical serverless functions [35] reveals that functions spend the majority of their overall execution time (89.7% on average) in the performance-sensitive library code, as shown in Fig. 4. The application entrypoint typically acts as a lightweight orchestration logic that initiates the compute-intensive tasks offloaded to external dependencies.

Additionally, when an application is packaged for deployment, dependencies dominate the overall size of the package. The storage component of Fig. 4 details the percentage of the total package size consumed by dependencies across the benchmark suite. While library footprint represents 43.5% of the total application package on average, for some benchmarks, it can take up to 98.2%.

It is important to note that it is the compute-intensive dependencies that predominantly benefit from CPU-specific optimizations. For example, libraries performing media processing or machine learning inference commonly utilize compiler intrinsics or handwritten assembly code targeting instruction set extensions (such as x86-64 AVX-512 or ARM NEON) to achieve high performance (see Fig. 2). In contrast, user code implementing the business logic rarely requires or benefits from hardware-specific optimizations. This structure, where the lightweight business logic is packaged with heavy and hardware-dependent libraries inside a single application package, serves as the primary motivation for our architectural design in the following section.

3 Mosaic

Mosaic is a novel cloud platform designed to meet the needs of compute-intensive elastic applications in heterogeneous environments. Mosaic redefines traditional application packaging by breaking down applications into lazily-bound *Tiles* to promote high utilization across heterogeneous machines without compromising CPU-specific optimizations. The proposed architecture consists of a runtime that implements lazy binding of performance-sensitive application components after the deployment target is known (§3.1) and a platform that optimizes data center utilization while still optimizing for developer-specified goals (§3.2). In §3.3, we describe how Mosaic can be used by modern hyperscalers.

3.1 Mosaic Runtime

The Mosaic runtime is the central component of the Mosaic ecosystem. The high-level overview of the runtime is visualized in Fig. 5. Its design leverages the finding discussed in §2.3: the total execution time of a typical cloud application is dominated by the code of its dependencies, e.g., libraries or external binaries (see Fig. 4) that can benefit from CPU-specific instruction set extensions. Therefore, we consider dependencies to be performance-critical, whereas user code acts as an entry point calling into the libraries. Furthermore, dependencies are application-agnostic and can be used by multiple different applications, while the user code is specific to each developer's needs.

Based on these insights, Mosaic splits cloud applications into *Tiles*, where each tile can be user code or a performance-critical library. User code tiles can import functions from other library or user code tiles. Contrary to the existing popular packaging approaches that eagerly package user code along with its dependencies into a single binary or container image, Mosaic considers applications as dependency graphs of lazily-bound tiles. For example, to deploy a function performing ML classification, developers provide two components: an entrypoint user code tile with business logic (reading an image and calling a `classify` function) and a *Tile Dependencies* file. This file is a JSON manifest describing which functions (e.g., `classify`) from which libraries (e.g., onnx) are invoked from their user code.

For portability purposes, Mosaic expects that user code tiles will be compiled into WebAssembly (Wasm) format. The library tiles are compiled into the native instruction set format of the CPUs available in the platform, including the CPU-specific optimizations. Since platforms have the full observability over their fleet, we expect platforms to compile commonly-used libraries, tailoring the compilation for each available machine type, and store them in an internal registry. Fig. 5 shows how this registry maintains library tiles tailored for specific microarchitectures. In addition, developers can provide their custom builds of the library tiles, for example, if some library is not available in the registry. This approach

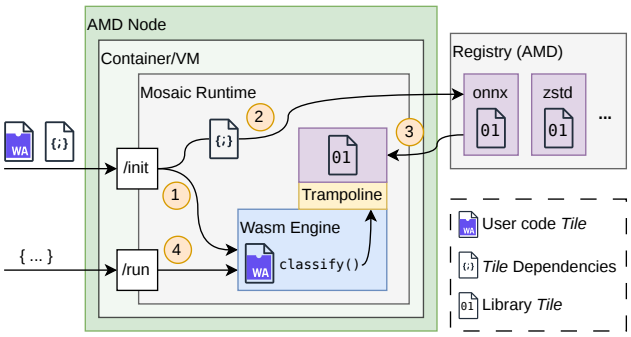


Figure 5. Deploying the Mosaic runtime on an AMD machine. First, the runtime initializes a Wasm engine for the user code tile (1). Then, it parses tile dependencies, pulls the corresponding library tile compiled specifically for this AMD machine (2), and injects it into the Wasm engine (3). After this, the runtime is ready to accept incoming invocations to the application (4).

allows developers to reap the benefits of CPU-binding (described in §2.2) at the level of individual libraries.

During the deployment of an application, the runtime creates a Wasm engine for the user code tile (step 1 in Fig. 5) and parses the tile dependencies file to pull the library tiles optimized for the target machine from the tile registry (step 2 in Fig. 5). Then, the runtime injects the requested functions from the libraries into the Wasm engine as host functions [26] (step 3 in Fig. 5). We describe the details and limitations of injecting host functions into the Wasm engine in §3.1.2. After this step, the Mosaic runtime instance is initialized and ready to execute the application code upon incoming requests (step 4 in Fig. 5).

In Mosaic, application packaging (i.e., linking the user code with the libraries) happens at deployment time, once the target machine is known. We refer to this approach as **lazy binding** as opposed to the eager-binding approaches discussed in §2.2. Lazy binding enables Mosaic to pick the dependency implementations optimized for the target deployment machine similarly to CPU-binding while offering cross-architecture portability thanks to Wasm, while ISA-binding is constrained to one architecture.

3.1.1 Runtime Interface.

We designed the Mosaic runtime as an HTTP server exposing a minimal interface. This interface is heavily inspired by OpenWhisk [2], an open-source serverless platform. It exposes two endpoints: **`/init`** which initializes the runtime instance with a given application (steps 1-3 in Fig. 5) and **`/run`** which invokes the application (step 4 in Fig. 5).

Once the runtime instance is initialized with an application by calling the `/init` endpoint, the `/run` endpoint can be called multiple times with different arguments since the

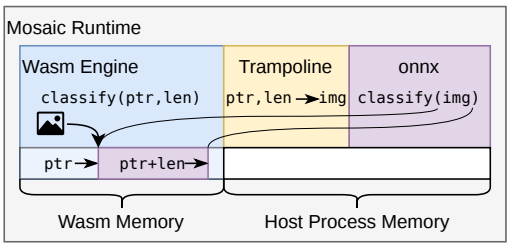


Figure 6. An example interaction of a user code tile (`classify` function) with a library tile (`onnx`) through a trampoline. The Wasm guest code passes a pointer to the image buffer and its length, the trampoline reconstructs an image object from the pointer and the length without copying data, and the library code is invoked with this image object, pointing to the original image buffer allocated in Wasm Memory.

runtime is cached for a configurable amount of time. Subsequent requests to the `/run` endpoint reuse the Wasm engine instance but create a new Wasm Memory [25], a boundary-checked linear memory region on which the Wasm engine can operate. This erases leftover memory from the previous executions and avoids memory fragmentation. As shown in §4.5, the overhead of initializing Wasm Memory for each request is negligible even for short-running applications.

3.1.2 Library Tile Trampoline. To execute applications, the Mosaic runtime must facilitate communication between the user code tiles (running in Wasm) and the native library tiles. We refer to the user code as guest code and to native code (including the Mosaic runtime and library code) as host code to highlight that the user code runs in an isolated Wasm environment. Bridging the guest-host boundary presents two technical challenges:

- (**C1**) Wasm natively supports only primitive numeric types (32-bit and 64-bit integers and floats) as host function arguments. High-level language constructs such as strings, arrays, or complex types cannot be passed as-is between a Wasm guest and a host function;
- (**C2**) Because Mosaic employs lazy binding, the runtime loads native libraries dynamically as shared objects. Consequently, the runtime cannot rely on static linking or compile-time type checking, and must interact with these libraries through a Foreign Function Interface (FFI).

To overcome these challenges, Mosaic requires each library tile to expose a *trampoline* facade (Fig. 6). The purpose of a trampoline is to act as an intermediate dispatch layer between the Wasm guest and the actual native code of the library. When user code invokes a library function, it cannot pass complex objects because of **C1**. Instead, it passes integer pointers to the objects (strings, arrays) in the linear Wasm Memory and the lengths of these objects to the host. The trampoline intercepts this FFI-compliant call, reads the raw bytes from the Wasm Memory, and reconstructs the high-level data types required by the library. Fig. 6 shows how this approach avoids copying data from Wasm to the host memory when passing an image to the `onnx` library performing ML classification.

Furthermore, to address the dynamic loading constraints outlined in **C2**, the trampoline provides a dispatch mechanism. To simplify dynamic linking of libraries containing functions with varying signatures, Mosaic expects library tiles to expose a single FFI entry point. This entry point accepts a target function identifier and an array of call arguments, and routes the execution to the actual library function. From the developer's perspective, user code calls a normal library function, as these pointer manipulations and dynamic function dispatch occur entirely on the trampoline side.

While our design introduces manual pointer reconstruction and function dispatch overhead to address **C1** and **C2**, it avoids expensive deep copies of data passed to and from library function invocations. In §4.5, we demonstrate that the trampoline overhead is negligible. Besides, through our adaptation of the SeBS [35] benchmarks in §4, we demonstrate that the proposed design accommodates the needs of typical performance-sensitive cloud applications.

3.2 Mosaic Platform

The Mosaic platform orchestrates the deployment and execution of applications across heterogeneous nodes, leveraging the lazy binding mechanism introduced in §3.1. As discussed in §2.2, CPU-binding limits deployment flexibility by constraining applications to specific machine types, whereas ISA-binding prevents hardware-specific optimizations. By decoupling the portable user code from the performance-critical dependencies, the Mosaic platform can schedule applications even across different CPU architectures while ensuring that each application still uses the libraries optimized for the target deployment machines. Fig. 7 illustrates the high-level architecture of the system.

The entry point to the platform is the global scheduler, which distributes incoming application invocations across the heterogeneous data center machines (nodes) it manages. As demonstrated previously in Fig. 3 and Tab. 1, there is no single optimal deployment configuration for pure performance and cost efficiency since the same workload may yield different performance on different CPUs. To accommodate pragmatic deployment preferences, the Mosaic platform allows developers to optionally provide an ordered list of preferred machine types along with the application. These preferences merely act as soft scheduling hints. Contrary to strict CPU-binding, if a preferred machine type is fully utilized, Mosaic allows the scheduler to fall back to the next available machine type in the list, avoiding the queuing delays and resource fragmentation. If a developer does not provide this list, the scheduler defaults to maximizing the data center utilization.

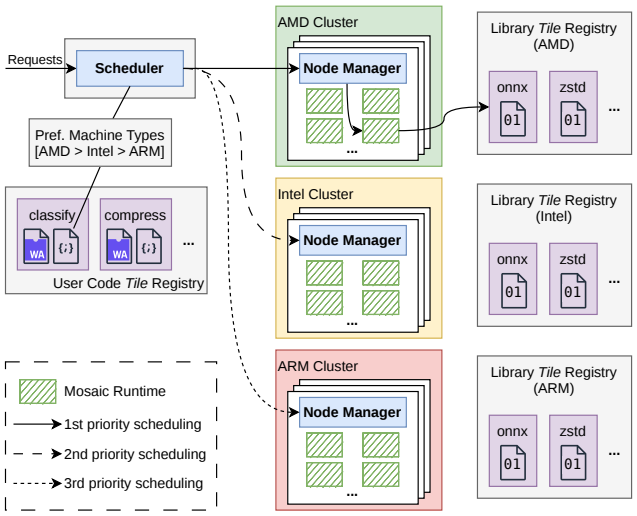


Figure 7. An example deployment of the Mosaic platform. The scheduler manages a data center divided into three clusters by CPU type. The Mosaic runtime fetches library tiles compiled specifically for the machine it is deployed on. The `classify` application specifies a list of preferred machine types, and the scheduler considers this list when deploying the application.

Each machine in the data center runs a node manager, a component responsible for managing local Mosaic runtime instances. The node manager handles invocations from the upstream global scheduler and forwards them to the Mosaic runtime instances. It also manages the lifecycle of the local Mosaic runtime instances, creating and terminating them according to the platform's resource management policies. To ensure proper tenant-level isolation, each runtime instance can be sandboxed in a container [12, 15] or a microVM [28].

Finally, the Mosaic platform maintains a tile registry that stores the user code tiles and pre-compiled library tiles optimized for the different machine types available in the data center. When the platform deploys an application, the runtime fetches its tiles tailored for the machine it is being deployed on. The library tiles served by this registry can be reused by multiple applications, naturally deduplicating storage occupied by popular libraries.

3.3 Mosaic in the Wild

The Mosaic runtime is implemented in Rust. It supports user code tiles written in any language that can be compiled into a WebAssembly Module [24]. The user code has to be adapted so that instead of making a direct call into the library, it calls an external host function. For example, in a user code tile written in Rust, this function has to be defined in the `unsafe extern "C"` block instead of being imported with a `use` statement. The tile dependencies file must list all host functions used by the user code. These changes require minimal developer effort and can be easily automated.

Due to the FFI-imposed limitations between the Wasm guest code and the dynamically-loaded native libraries (discussed in §3.1.2), Mosaic does not natively support library functions that have high-level language constructs such as `struct` in their signature. Passing such data types can be made possible by (de-)serializing the respective objects, which can be facilitated with code generation. Currently, Mosaic supports passing primitives, strings, and arrays of primitives between user code and library tiles without copying or serializing the data.

In case developers want to use a library currently not supported by the platform or include CPU-specific optimizations in their code, they can prepare their own library tile. Creating a library tile does not require changes to the library's source code but instead involves writing a trampoline that exposes the native library's public API to the Wasm module. Out of 14 library functions we adapted for the benchmarks, nine required manual intervention to convert structured Rust types into Wasm-compatible types, and the rest required only simple pointer-to-object reconstruction logic that could be automatically generated. For the functions requiring manual intervention, the average number of lines of code written manually is 24. Since writing trampolines is mostly a mechanical, low-effort task, it should be practical for platforms and developers to create and maintain library tiles.

Our solution primarily focuses on applications written in natively compiled languages such as Rust or C. These languages are the standard choice for performance-critical applications, but their compiled binaries lack portability. With Mosaic, we offer cross-architecture portability to applications written in these languages without sacrificing their native execution performance. Furthermore, while popular interpreted or managed languages such as Python or Java are inherently portable, they often rely on native libraries like NumPy [16] or TensorFlow [27] to execute performance-critical operations. This reliance on native dependencies forces the entire application to be bound to a specific CPU or ISA. Mosaic's lazy-binding design can be extended to these languages, allowing their native libraries to be dynamically injected by the platform at deployment time.

For Wasm code execution, the Mosaic runtime uses Wasmtime [23]. To provide host functionality to applications, we use WASI [22], a standard server-side system interface specification for Wasm runtimes. This system interface can be further extended with a solution like WALI [46].

4 Evaluation

In this section, we aim to answer the following questions:

- How does Mosaic affect serverless scheduling based on a real-world trace, and what is its impact on end-to-end latency? (§4.2)

- How does Mosaic compare to eager-binding approaches in pure performance expressed as throughput? (§4.3)
- What are the storage requirements for serverless functions using Mosaic packaging compared to the other application packaging approaches? (§4.4)
- What is the overhead of the Mosaic's lazy binding? (§4.5)

4.1 Evaluation Methodology

Benchmarks. We use benchmarks from SeBS [35], an established benchmark suite for serverless. We adapted these benchmarks to Rust. For Mosaic, we extracted dependencies in separate library tiles with trampolines and compiled the user code into WebAssembly.

Comparison baselines. We compare *Mosaic* to *CPU-binding* (each benchmark compiled with optimizations specific to the target CPU, e.g., using `target-cpu=native` flag) and *ISA-binding* (each benchmark compiled without forcing advanced CPU-specific instructions). For storage footprint and startup latency measurements, we include two additional baselines: i) *XaaS* [34], which delays application (and library code) compilation until the target machine is known; and ii) *uber-packaging*, which requires developers to build each application for every machine type.

Experimental setup. We evaluate the throughput and latency of Mosaic and the eager-binding baselines (§4.3) across three AWS EC2 [7] machine types to mimic the heterogeneous hardware landscape present in modern clouds:

- **C7i** [5] equipped with the 4th generation Intel Xeon Scalable CPU @ 3.2 GHz;
- **C7a** [3] equipped with the 4th generation AMD EPYC CPU @ 3.7 GHz;
- **C7g** [4] equipped with AWS Graviton3 CPU @ 2.6 GHz.

All machines are provisioned as `4xlarge` instances, featuring 16 vCPUs and 32 GB of DDR5 memory and run Ubuntu 22.04. We run the scheduler experiment (§4.2), cold-start latency experiment (§4.4), and Mosaic execution breakdown (§4.5) on a local cluster node equipped with two Intel Xeon Gold 6530 CPUs @ 2.1–4 GHz and 128 GB of DDR5 memory, running Debian 13, Linux kernel 6.12.41.

4.2 Latency and Cost Impact of Lazy Binding

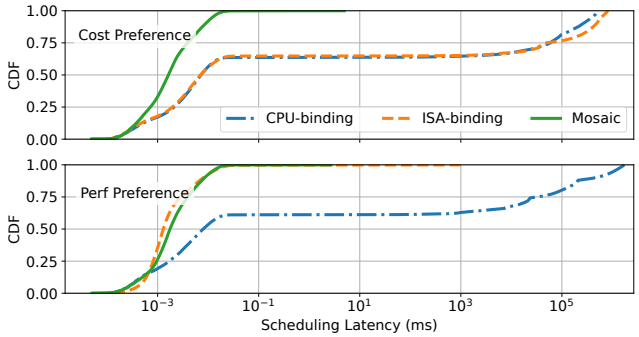
Data center-level scheduling introduces resource constraints that impact end-to-end function invocation latency. We implement a realistic serverless environment to evaluate Mosaic's impact on data center-wide scheduling by using the public Azure Functions trace [47].

4.2.1 Experimental Platform Setup. We extract a full 1-hour long trace segment (15th hour of day 2) from the Azure Functions trace [47] containing 42.4 million invocations to 23.6 thousand unique functions. We map functions to the 10 benchmarks from our suite based on their average durations, adopting a methodology used by recent studies in serverless [36, 41]. We construct a virtual data center consisting of 100 nodes managed by the global scheduler, maintaining a proportional distribution of machine types based on the Cloudspecs study [48]: 53% of Intel (C7i) instances, 17% of AMD (C7a) instances, and 30% of AWS Graviton (C7g) instances. The scheduler dispatches function invocations to these virtual nodes that emulate execution times by returning responses after the exact duration recorded for that specific benchmark, machine type, and execution mode combination from our microbenchmarks (§4.3). We run this experiment for two pragmatic profiles: a *Cost Preference* scenario, where developers request the platform to optimize for cost-efficiency, and a *Performance Preference* scenario, where developers request the platform to optimize for performance. These hardware preferences for each benchmark and scenario are taken from Tab. 1. We note that while we produced Tab. 1 empirically, platforms could achieve the same by combining developer hints and incrementally detecting the best hardware configuration from historical data for each library.

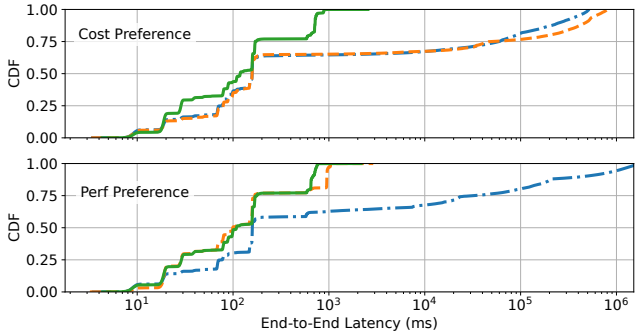
We exclude the XaaS and uber-packaging baselines from the scheduling experiment because both approaches, similarly to Mosaic, allow executing workloads on any available machine types. However, these approaches achieve this flexibility at a prohibitive cost for serverless workloads. Specifically, XaaS imposes severe cold-start overheads due to compiling applications at deployment time on the target machine, and uber-packaging inflates storage because of providing pre-compiled binaries for all possible deployment targets. We compare Mosaic with these systems to quantify their storage and cold-start overheads in §4.4.

4.2.2 Cost Preference. For the cost-optimizing scenario, Tab. 1 indicates that 4 out of 10 benchmarks request ARM machines (C7g). However, because C7g nodes constitute only 30% of our virtual data center, these resources are rapidly depleted. Consequently, both CPU-binding and ISA-binding suffer from lower utilization caused by data center fragmentation. As shown in Fig. 8a (upper subplot), function invocations are forced to wait in queues for their requested hardware to become available. At the 90th percentile, scheduling latency reaches 286–433 seconds for the eager-binding baselines, leading to significant increase in end-to-end latencies (Fig. 8b, upper subplot). Mosaic with its cross-architecture portability avoids isolating workloads in the partitioned clusters, bringing the scheduling latency to a sub-millisecond scale. By avoiding enqueuing requests, Mosaic reduces end-to-end invocation latency in the 90th percentile by 399–604× compared to CPU-binding and ISA-binding, respectively.

4.2.3 Performance Preference. For the second scenario (performance-optimizing), five benchmarks request Intel (C7i), and five benchmarks request AMD (C7a) machines. This distribution penalizes CPU-binding because the AMD-preferring functions are bottlenecked by the small number of AMD machines (which constitute 17% of the virtual data center). This results in 404 seconds of scheduling latency in



(a) Scheduling latency.



(b) Total request latency.

Figure 8. Replaying the 1-hour invocation trace from Azure Functions [47] and measuring scheduling latency (a) and end-to-end request latency (b). In total, the trace contains 42,376,709 invocations of 23,598 unique functions.

the 90th percentile for CPU-binding (Fig. 8a, lower subplot). In contrast, ISA-binding targets both C7i and C7a machines (combined 70% of the virtual data center), as they belong to the same architecture. Therefore, in performance preference scenario, both ISA-binding and Mosaic experience sub-millisecond scheduling latency, and Mosaic improves end-to-end latency in the 90th percentile by 595× compared to CPU-binding (Fig. 8b, lower subplot). Additionally, Mosaic improves end-to-end latency over ISA-binding by 1.4× due to shorter execution times of library tiles optimized for target CPUs, whereas ISA-binding cannot benefit from CPU-specific optimizations.

Summary. The results in Fig. 8 highlight that Mosaic outperforms eager-binding approaches when there is a mismatch between resource demand and the actual hardware availability. The long queuing delays observed for the early-binding approaches represent the real-world penalty that stems from the lack of deployment flexibility. Because Mosaic offers cross-architecture portability, it minimizes the queueing time of requests by being able to schedule them on idle machines. In sum, Mosaic is able to sustain the same throughput and avoid latency spikes using less overall infrastructure compared to eager-binding approaches.

Table 2. Cost of Mosaic and ISA-binding execution normalized to CPU-binding. The cost was measured for the 1-hour trace using per-hour prices of AWS C7i, C7a, and C7g instances in size `xlarge`.

Mode	Cost Preference	Perf Preference
Mosaic	1.16	1.36
ISA-binding	1.01	1.67

4.2.4 The Cost of Flexibility.

While Mosaic eliminates scheduling bottlenecks imposed by cluster partitioning, it may introduce cost and performance unpredictability. When Mosaic scheduler reschedules a workload to a non-preferred machine type to avoid queuing, the execution time or the price rate of the fallback machine may be higher than the developer's expectations. We quantified the overall cost of the experiment for each mode and scenario across all machines by multiplying the total execution time spent in each virtual node by the cost of the respective instance on AWS in size `xlarge`. The results in Tab. 2 show that Mosaic and ISA-binding introduce cost unpredictability as these modes can schedule workloads on different CPUs with different pricing rate. However, Mosaic reduces end-to-end latency compared to both baselines regardless of developer preferences, avoiding severe service-level agreement (SLA) breaches which may oblige the platforms to compensate the costs to the developers. To limit financial unpredictability, platforms can allow developers to specify a budget cap per invocation, explicitly preventing the scheduler from falling back to prohibitively expensive hardware. Ultimately, cloud providers can also mask this unpredictability by charging a fixed rate per invocation while leveraging Mosaic's portability to maximize overall data center utilization behind the scenes. We note that designing pricing models for Mosaic is out of the scope of this work.

4.3 Quantifying the Overhead of Portability

This experiment focuses on pure performance characteristics of Mosaic compared to CPU-binding and ISA-binding. We measure throughput as a target metric for this experiment, and collect latency measurements for the experiment discussed in §4.2. We assume a traditional serverless scenario, where each runtime instance serves at most one request at a time, and we use CPU affinity options to bind each runtime instance to a core. Each runtime instance runs inside a Docker container limited to using 1 CPU and 2 GB of memory.[2] Throughput is measured using Apache Bench.[3] The

[2]This ratio is inspired by a proportional core-to-memory ratio used in Amazon Lambda [6].
[3]https://httpd.apache.org/docs/2.4/programs/ab.html

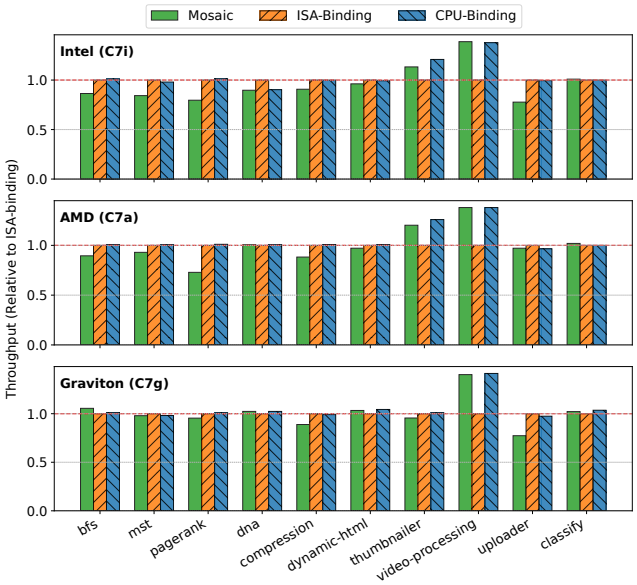


Figure 9. Throughput comparison of the Mosaic runtime, ISA-binding, and CPU-binding for different benchmarks on heterogeneous machines, normalized to ISA-binding.

results are presented in Fig. 9, each subplot shows throughput increase or degradation relative to ISA-binding for each machine type.

Intel and AMD. Fig. 9 shows that on the Intel and AMD machines, which share the x86-64 architecture, compute-intensive benchmarks strictly benefit from CPU-specific optimizations offered by Mosaic and CPU-binding. Specifically, `thumbnailer` and `video-processing` achieve up to 20.2–38.8% increase in throughput in Mosaic and up to 28.9–38% with CPU-binding compared to ISA-binding. We attribute this increase in throughput to the use of advanced x86-64 SIMD instructions, such as AVX2 and AVX-512, that these benchmarks benefit from the most. In particular, the `video-processing` benchmark uses FFmpeg, which features handwritten assembly code with vectorized instructions for performance-sensitive code paths.

Conversely, for graph-processing benchmarks such as `bfs`, `mst`, and `pagerank`, whose implementation is primarily composed of pointer chasing, or `uploader`, which is I/O-bound, Mosaic exhibits performance on par with CPU-binding and ISA-binding or degradation of up to 27.1%. For these benchmarks, advanced vectorization instructions do not offer any benefits, but Mosaic still incurs the overhead of Wasm execution and FFI boundary crossing (detailed in §4.5). Also, the use of wide vectorization instructions inherently introduces the overhead of constant gather/scatter operations (moving elements from non-contiguous memory addresses into vector registers and back). Finally, issuing AVX-512 instructions on modern x86 processors can trigger core frequency throttling to prevent overheating [38, 39]. This throttling also affects the scalar instructions, resulting in overall performance loss for applications that do not perform dense arithmetic operations.

AWS Graviton. The bottom part of Fig. 9 shows that the compute-intensive `video-processing` benchmark yields up to 40.2% throughput increase for Mosaic and 41.5% increase for CPU-binding compared to ISA-binding, for the same reasons as for x86-64 machines. Similar to x86-64 results, Mosaic on AWS Graviton shows up to 22.7% degradation for `uploader`. At the same time, for the rest of the benchmarks, there is no significant degradation or improvement for both Mosaic and CPU-binding compared to ISA-binding, unlike in x86-64 machines. The ARM's vectorization extensions do not impose the same aggressive frequency throttling as observed with AVX-512 on x86-64 [10], thus not penalizing the overall application execution.

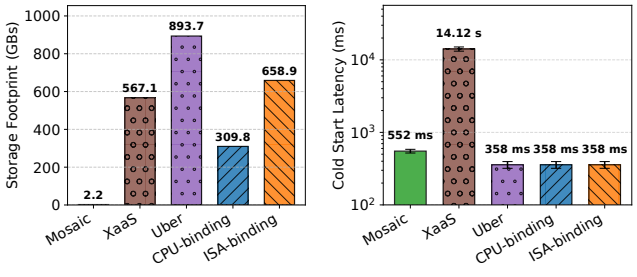
Summary. The results in Fig. 9 demonstrate that Mosaic is best suited for performance-sensitive workloads that can be optimized through the extensive use of the recent CPU-specific extensions. For the majority of the benchmarks, Mosaic performs on par with eager-binding baselines across all machine types. For short-running non-compute-intensive benchmarks, where the overhead of Mosaic's design becomes more acute, Mosaic shows performance degradation compared to ISA-binding. However, as discussed in §4.2, the portability of Mosaic compensates for performance degradation by allowing the platform to schedule workloads in partitioned heterogeneous clusters without waiting for specific machine types to become available when fully utilized. Therefore, Mosaic can serve as a strong default for most serverless deployments.

4.4 Comparing Solutions for Portability

To fully evaluate the Mosaic's design, we quantify the overheads Mosaic introduces to the cold starts and discuss the overall storage requirements for applications packaged for Mosaic compared to CPU-binding, ISA-binding, XaaS, and uber-packaging baselines.

Storage Requirements. To evaluate the impact of Mosaic's packaging format, we calculate the storage requirements of the 23,598 functions from the 1-hour Azure trace for each baseline's packaging approach (Fig. 10a). The numbers in Fig. 10a only account for binary sizes, without considering container image overhead.

For CPU-binding and ISA-binding, each function is packaged as a native binary with all dependencies included, resulting in a total storage footprint of 309.8 GB and 658.9 GB, respectively. This difference between CPU-binding and ISA-binding stems primarily from the size of the FFmpeg binary — the CPU-specific binary is 3.13× smaller than the generic binary on average across the machine types considered (21.38 MB vs. 67.01 MB). Uber-packaging exacerbates this overhead by bundling CPU-bound function binaries for



(a) Storage footprint. **(b)** Cold-start latency.

Figure 10. Comparison of packaging approaches. (a) Total storage footprint of all functions used in the 1-hour Azure Functions trace. Mosaic reduces storage footprint by sharing common library tiles and by compiling user code tiles into lightweight Wasm format. (b) Startup time latency of Mosaic, XaaS, and eager-binding approaches. The eager-binding approaches share the same cold-start latency because they follow the same startup sequence.

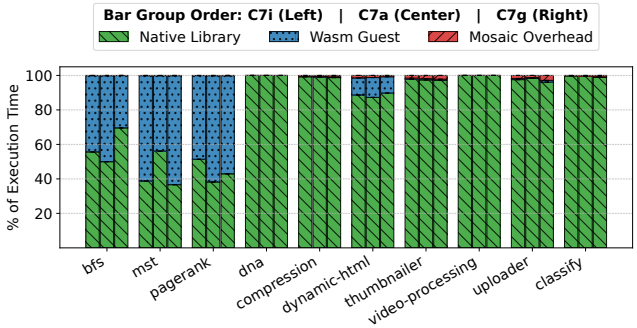


Figure 11. Breakdown of execution time spent in library code (Native Library), user code (Wasm Guest), and Wasm preparation phases or trampoline glue code (Mosaic Overhead). Mosaic benefits the most from long-running CPU-optimized library logic.

all three machine types (C7i, C7a, C7g), increasing the storage footprint to 893.7 GB. Various storage deduplication techniques [31, 44, 49, 51] can be applied to mitigate the duplication of common dependencies across functions. However, in popular performance-oriented languages such as Rust and Go, applications are typically statically linked into a single monolithic binary, making deduplication non-trivial.

The XaaS approach, which stores source code to be built at deployment time, requires 567.1 GB. While the source code of custom user logic is typically minimal (8.06 KB on average across our benchmarks), XaaS requires bundling the source code of performance-critical external dependencies (such as the FFmpeg codebase) for every function that relies on them, leading to storage bloat.

In contrast, Mosaic requires only 2.2 GB of total storage for the same functions. This footprint reduction is achieved primarily through the decoupling of user code from its dependencies and storing the native library tiles in the platform registry once per each machine type. The library tiles can then be safely shared across different applications. Additionally, the user code tiles, compiled into Wasm and stored individually for each function, are compact and have an average size of 92.69 KB across our benchmark suite. Consequently, Mosaic reduces storage footprints by over two orders of magnitude compared to the baselines.

Cold Starts. To evaluate Mosaic's lazy binding overhead, we measure the average cold-start latency of preparing a function instance to serve its first request. For this experiment, we use the `compression` benchmark as a representative workload, which relies on two library tiles.

The initialization procedure varies depending on the packaging approach, resulting in different cold-start latencies (Fig. 10b). For the XaaS approach, a cold start entails starting up a Docker container, downloading the source code, and compiling the function inside the Docker container. The compilation process incurs an average cold start of 14.12 s, which is impractical for serverless environments where cold-start latency is critical. While the compilation artifact can be cached for future reuse, an optimal XaaS deployment where all compilation artifacts are eventually cached functionally converges into uber-packaging with high storage footprint.

In contract, eager-binding approaches (CPU-binding, ISA-binding, and uber-packaging) achieve the lowest average cold-start latency at 358 ms. Eager-binding bundles the user code and its dependencies into a single native binary. Therefore, for these approaches, the cold-start process consists of starting a serverless runtime container, downloading the binary, and dynamically loading it into memory.

Mosaic performs lazy binding on each cold start, which includes booting the Mosaic runtime container, fetching the user code tile and creating a Wasm engine for it, downloading the requested library tiles from the registry, and injecting the host functions into the Wasm engine. The average cold start for Mosaic is 552 ms. While Mosaic's lazy binding introduces the ~194 ms overhead compared to eager-binding, we consider this one-time penalty to be an acceptable trade-off for the cross-architecture portability (§4.2) and reduction in storage footprint (Fig. 10a) that Mosaic provides. We leave optimizations such as concurrent tile retrieval and injection, Wasm engine initialization, and local library tile cache prewarming for future work.

4.5 Runtime Execution Overhead

Mosaic splits functions into user code tiles running as Wasm guests and library tiles which are natively compiled with CPU-specific optimizations. A Wasm guest calls into a native library through a trampoline. To measure the overhead of this design, we instrument the Mosaic runtime to record

the time spent in three execution phases: the execution of the user code within the Wasm sandbox (Wasm Guest), the execution of the hardware-optimized library code (Native Library), and the Mosaic overhead. This overhead encompasses Wasm Memory [25] initialization and trampoline pointer manipulation across the FFI boundary required to reconstruct high-level data types.

Fig. 11 shows that the Mosaic overhead is negligible across the benchmarks compared to the overall execution time. This proves that the design of our trampoline efficiently bridges that Wasm-to-native boundary without imposing serialization or copying data even for data-intensive benchmarks.

Furthermore, the execution breakdown validates the core insight discussed in §2.3: for the majority of the benchmarks considered, the total execution time is dominated by the library tiles. Because Mosaic ensures these libraries are compiled with optimal CPU-specific flags, the overall application performance remains highly competitive with static CPU-binding. Graph-processing benchmarks (`bfs`, `mst`, `pagerank`) are an exception to this observation, spending more than 50% of total execution time on average running the user code in Wasm. While these benchmarks use natively compiled library tiles for their core functionality, they generate the graph data structures in the user code (Wasm Guest). This prolonged execution in Wasm, combined with the vectorization penalties discussed in §4.3, explains the performance degradation for these benchmarks.

5 Related Work

Hardware-Specific Reconfiguration/Packaging. A recent work, XaaS [34], proposes delaying the compilation of HPC applications until the deployment target is known and building the application executable from source or intermediate representation (IR) directly on the target machine. However, compiling an application at deployment time leads to high cold-start latencies, making such an approach impractical for serverless platforms. Another way to take advantage of hardware heterogeneity is to bundle an application with multiple binaries for all possible targets. Popular libraries [13, 17, 20] often offer pre-built binaries for different architectures that developers can include in their application packages. However, such an approach leads to storage bloat. Mosaic allows developers to benefit from hardware heterogeneity without inflating storage or incurring compilation overhead during cold starts.

JIT Compilation and Heterogeneous Execution. Managed language runtimes frequently utilize Just-In-Time (JIT) compilation to dynamically optimize portable bytecode for the underlying hardware at execution time. TornadoVM [37] dynamically compiles Java bytecode to utilize CPUs, GPUs, and FPGAs. Similarly, Numba [43] provides accelerator-aware JIT compilation for Python, and programming models such as SYCL [19] abstract hardware heterogeneity from developers while utilizing available hardware accelerators. Mosaic focuses on natively compiled applications utilizing performance-sensitive libraries, such as FFmpeg [13] or OpenSSL [18]. These libraries commonly contain hand-written assembly code targeting specific CPU extensions (e.g., AVX-512 or ARM NEON) in performance-critical code paths, which cannot be easily inferred or generated by JIT compiler heuristics. Furthermore, recent studies [32, 42] demonstrate that JIT compilation introduces non-negligible startup latency and execution overhead in serverless environments. By relying on pre-compiled native libraries, Mosaic avoids this overhead, making it suitable for short-lived, latency-sensitive serverless functions.

Storage Deduplication. Extensive research has focused on storage deduplication solutions. AWS Lambda features container layer deduplication [31], research works such as DADI [44], FaaSNet [51], and Cntr [49] focus on deduplicating container images to reduce startup times. These solutions perform active deduplication, requiring additional compute overhead to continuously analyze container data. In contrast, Mosaic offers deduplication at an architectural level by decoupling lightweight custom application logic from common dependencies that can be shared by multiple applications, reducing the overall storage footprint across the platform without requiring an active deduplication infrastructure. Still, Mosaic is orthogonal to block-based deduplication techniques, which can still be applied to further compress the platform's tile registries.

6 Conclusion

Mosaic proposes decoupling serverless applications into portable business logic and performance-critical native dependencies, enabling platforms to lazily bind workloads to the most optimal available hardware at deployment time. Our evaluation demonstrates that this approach allows compute-intensive workloads to achieve near-native performance by fully utilizing CPU-specific extensions, while simultaneously preserving deployment flexibility and improving overall data center utilization.

Acknowledgments

We thank the anonymous reviewers for their insightful comments and suggestions. This work was supported by Fundação para a Ciência e a Tecnologia I.P. (FCT), under grant 2024.01902.BD (DOI: https://doi.org/10.54499/2024.01902.BD), and projects LISBOA2030-FEDER-00748300 (DOI: https://doi.org/10.54499/2023.16994.ICDT), UID/50021/2025 (DOI: https://doi.org/10.54499/UID/50021/2025), and UID/PRR/50021/2025 (DOI: https://doi.org/10.54499/UID/PRR/50021/2025).

References

[1] 2020. Processing user-generated content using AWS Lambda and FFmpeg. https://aws.amazon.com/blogs/media/processing-user-generated-content-using-aws-lambda-and-ffmpeg. Accessed: 2026-04-09.

[2] 2022. OpenWhisk - Open Source Serverless Cloud Platform. https://openwhisk.apache.org/. Accessed: 2026-06-04.

[3] 2026. Amazon EC2 C7a instances. https://aws.amazon.com/ec2/instance-types/c7a/. Accessed: 2026-06-07.

[4] 2026. Amazon EC2 C7g instances. https://aws.amazon.com/ec2/instance-types/c7g/. Accessed: 2026-06-07.

[5] 2026. Amazon EC2 C7i instances. https://aws.amazon.com/ec2/instance-types/c7i/. Accessed: 2026-06-07.

[6] 2026. Amazon Lambda Quotas. https://docs.aws.amazon.com/lambda/latest/dg/gettingstarted-limits.html. Accessed: 2026-06-07.

[7] 2026. AWS EC2. https://aws.amazon.com/ec2. Accessed: 2026-04-09.

[8] 2026. AWS EC2 Spot Instances. https://aws.amazon.com/ec2/spot. Accessed: 2026-04-09.

[9] 2026. AWS Graviton Technical Guide. https://github.com/aws/aws-graviton-getting-started. Accessed: 2026-06-07.

[10] 2026. AWS Graviton3 delivers leading AES-GCM encryption performance. https://developer.arm.com/community/arm-community-blogs/b/servers-and-cloud-computing-blog/posts/aes-gcm-optimizations-for-armv8-4-on-neoverse-v1-graviton3. Accessed: 2026-06-07.

[11] 2026. Azure Spot Virtual Machines. https://azure.microsoft.com/en-us/products/virtual-machines/spot. Accessed: 2026-04-09.

[12] 2026. Docker. https://www.docker.com/. Accessed: 2026-06-04.

[13] 2026. FFmpeg. https://ffmpeg.org/. Accessed: 2026-06-07.

[14] 2026. Google Cloud Spot VMs. https://cloud.google.com/solutions/spot-vms. Accessed: 2026-04-09.

[15] 2026. gVisor. https://gvisor.dev/. Accessed: 2026-06-04.

[16] 2026. NumPy. https://numpy.org. Accessed: 2026-06-04.

[17] 2026. OpenBLAS. https://www.openmathlib.org/OpenBLAS/. Accessed: 2026-06-07.

[18] 2026. OpenSSL. https://openssl.org/. Accessed: 2026-06-07.

[19] 2026. SYCL - C++ Programming for Heterogeneous Parallel Computing. https://www.khronos.org/sycl/. Accessed: 2026-06-07.

[20] 2026. TensorFlow. https://www.tensorflow.org/. Accessed: 2026-06-07.

[21] 2026. TensorFlow not bundled with SSE. https://github.com/tensorflow/tensorflow/issues/7778. Accessed: 2026-06-07.

[22] 2026. WASI. https://wasi.dev/. Accessed: 2026-06-04.

[23] 2026. Wasmtime. https://wasmtime.dev/. Accessed: 2026-06-04.

[24] 2026. WebAssembly Modules. https://webassembly.github.io/spec/core/exec/modules.html. Accessed: 2026-06-04.

[25] 2026. WebAssembly Modules — Memories. https://webassembly.github.io/spec/core/syntax/modules.html#syntax-mem. Accessed: 2026-06-04.

[26] 2026. WebAssembly Runtime Structure — Function Instances. https://webassembly.github.io/spec/core/exec/runtime.html#function-instances. Accessed: 2026-06-04.

[27] Martín Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, Manjunath Kudlur, Josh Levenberg, Rajat Monga, Sherry Moore, Derek G. Murray, Benoit Steiner, Paul Tucker, Vijay Vasudevan, Pete Warden, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng. 2016. TensorFlow: a system for large-scale machine learning. In *Proceedings of the 12th USENIX Conference on Operating Systems Design and Implementation* (Savannah, GA, USA) *(OSDI'16)*. USENIX Association, USA, 265—-283.

[28] Alexandru Agache, Marc Brooker, Alexandra Iordache, Anthony Liguori, Rolf Neugebauer, Phil Piwonka, and Diana-Maria Popa. 2020. Firecracker: Lightweight Virtualization for Serverless Applications. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*. USENIX Association, Santa Clara, CA, 419–434. https://www.usenix.org/conference/nsdi20/presentation/agache

[29] Omid Alipourfard, Hongqiang Harry Liu, Jianshu Chen, Shivaram Venkataraman, Minlan Yu, and Ming Zhang. 2017. CherryPick: Adaptively Unearthing the Best Cloud Configurations for Big Data Analytics. In *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17)*. USENIX Association, Boston, MA, 469–482. https://www.usenix.org/conference/nsdi17/technical-sessions/presentation/alipourfard

[30] Pradeep Ambati, Inigo Goiri, Felipe Frujeri, Alper Gun, Ke Wang, Brian Dolan, Brian Corell, Sekhar Pasupuleti, Thomas Moscibroda, Sameh Elnikety, Marcus Fontoura, and Ricardo Bianchini. 2020. Providing SLOs for Resource-Harvesting VMs in Cloud Platforms. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. USENIX Association, 735–751. https://www.usenix.org/conference/osdi20/presentation/ambati

[31] Marc Brooker, Mike Danilov, Chris Greenwood, and Phil Piwonka. 2023. On-demand Container Loading in AWS Lambda. In *2023 USENIX Annual Technical Conference (USENIX ATC 23)*. USENIX Association, Boston, MA, 315–328. https://www.usenix.org/conference/atc23/presentation/brooker

[32] Joao Carreira, Sumer Kohli, Rodrigo Bruno, and Pedro Fonseca. 2021. From warm to hot starts: leveraging runtimes for the serverless era. In *Proceedings of the Workshop on Hot Topics in Operating Systems* (Ann Arbor, Michigan) *(HotOS '21)*. Association for Computing Machinery, New York, NY, USA, 58–64. doi:10.1145/3458336.3465305

[33] Andrew Chung, Jun Woo Park, and Gregory R. Ganger. 2018. Stratus: cost-aware container scheduling in the public cloud. In *Proceedings of the ACM Symposium on Cloud Computing* (Carlsbad, CA, USA) *(SoCC '18)*. Association for Computing Machinery, New York, NY, USA, 121–134. doi:10.1145/3267809.3267819

[34] Marcin Copik, Eiman Alnuaimi, Alok Kamatar, Valerie Hayot-Sasson, Alberto Madonna, Todd Gamblin, Kyle Chard, Ian Foster, and Torsten Hoefler. 2025. XaaS Containers: Performance-Portable Representation With Source and IR Containers. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '25)*. Association for Computing Machinery, New York, NY, USA, 533–555. doi:10.1145/3712285.3759868

[35] Marcin Copik, Grzegorz Kwasniewski, Maciej Besta, Michal Podstawski, and Torsten Hoefler. 2021. SeBS: A Serverless Benchmark Suite for Function-as-a-Service Computing. In *Proceedings of the 22nd International Middleware Conference* (Québec city, Canada) *(Middleware '21)*. Association for Computing Machinery, New York, NY, USA, 64–78. doi:10.1145/3464298.3476133

[36] Lazar Cvetković, François Costa, Mihajlo Djokic, Michal Friedman, and Ana Klimovic. 2024. Dirigent: Lightweight Serverless Orchestration. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles* (Austin, TX, USA) *(SOSP '24)*. Association for Computing Machinery, New York, NY, USA, 369–384. doi:10.1145/3694715.3695966

[37] Juan Fumero, Michail Papadimitriou, Foivos S. Zakkak, Maria Xekalaki, James Clarkson, and Christos Kotselidis. 2019. Dynamic Application Reconfiguration on Heterogeneous Hardware.. In *Proceedings of the 15th ACM SIGPLAN/SIGOPS International Conference on Virtual Execution Environments (VEE '19)*. Association for Computing Machinery. doi:10.1145/3313808.3313819

[38] Mathias Gottschlag and Frank Bellosa. 2018. Mechanism to Mitigate AVX-Induced Frequency Reduction. arXiv:1901.04982 [cs.DC] https://arxiv.org/abs/1901.04982

[39] Mathias Gottschlag, Peter Brantsch, and Frank Bellosa. 2020. Automatic Core Specialization for AVX-512 Applications. In *Proceedings of the 13th ACM International Systems and Storage Conference* (Haifa, Israel) *(SYSTOR '20)*. Association for Computing Machinery, New York, NY, USA, 25–35. doi:10.1145/3383669.3398282

[40] Kenta Ishiguro, Caleb Fonyuy-Asheri, Elouan Barraud, Renaud Lachaize, Yérom-David Bromberg, and Alain Tchana. 2026. Everything You Need to Know About Virtual Machine Live Migration Between Heterogeneous Processors. In *Proceedings of the 21st European Conference on Computer Systems* (Edinburgh, Scotland UK) *(EUROSYS '26)*. Association for Computing Machinery, New York, NY, USA, 1399–1413. doi:10.1145/3767295.3803609

[41] Serhii Ivanenko, Vasyl Lanko, Rudi Horn, Vojin Jovanovic, and Rodrigo Bruno. 2026. Hydra: Virtualized Multi-Language Runtime for High-Density Serverless Platforms. In *Proceedings of the 2025 ACM Symposium on Cloud Computing (SoCC '25)*. Association for Computing Machinery, New York, NY, USA, 644–658. doi:10.1145/3772052.3772267

[42] Sumer Kohli, Shreyas Kharbanda, Rodrigo Bruno, Joao Carreira, and Pedro Fonseca. 2024. Pronghorn: Effective Checkpoint Orchestration for Serverless Hot-Starts. In *Proceedings of the Nineteenth European Conference on Computer Systems* (Athens, Greece) *(EuroSys '24)*. Association for Computing Machinery, New York, NY, USA, 298–316. doi:10.1145/3627703.3629556

[43] Siu Kwan Lam, Antoine Pitrou, and Stanley Seibert. 2015. Numba: a LLVM-based Python JIT compiler. In *Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC* (Austin, Texas) *(LLVM '15)*. Association for Computing Machinery, New York, NY, USA, Article 7, 6 pages. doi:10.1145/2833157.2833162

[44] Huiba Li, Yifan Yuan, Rui Du, Kai Ma, Lanzheng Liu, and Windsor Hsu. 2020. DADI: Block-Level Image Service for Agile and Elastic Application Deployment. In *2020 USENIX Annual Technical Conference (USENIX ATC 20)*. USENIX Association, 727–740. https://www.usenix.org/conference/atc20/presentation/li-huiba

[45] Orestis Polychroniou and Kenneth A. Ross. 2015. Efficient Lightweight Compression Alongside Fast Scans. In *Proceedings of the 11th International Workshop on Data Management on New Hardware* (Melbourne, VIC, Australia) *(DaMoN'15)*. Association for Computing Machinery, New York, NY, USA, Article 9, 6 pages. doi:10.1145/2771937.2771943

[46] Arjun Ramesh, Tianshu Huang, Ben L. Titzer, and Anthony Rowe. 2025. Empowering WebAssembly with Thin Kernel Interfaces. In *Proceedings of the Twentieth European Conference on Computer Systems* (Rotterdam, Netherlands) *(EuroSys '25)*. Association for Computing Machinery, New York, NY, USA, 1–20. doi:10.1145/3689031.3717470

[47] Mohammad Shahrad, Rodrigo Fonseca, Inigo Goiri, Gohar Chaudhry, Paul Batum, Jason Cooke, Eduardo Laureano, Colby Tresness, Mark Russinovich, and Ricardo Bianchini. 2020. Serverless in the Wild: Characterizing and Optimizing the Serverless Workload at a Large Cloud Provider. In *2020 USENIX Annual Technical Conference (USENIX ATC 20)*. USENIX Association, 205–218. https://www.usenix.org/conference/atc20/presentation/shahrad

[48] Till Steinert, Maximilian Kuschewski, and Viktor Leis. 2026. Cloudspecs: Cloud Hardware Evolution Through the Looking Glass. In *CIDR*. www.cidrdb.org.

[49] Jörg Thalheim, Pramod Bhatotia, Pedro Fonseca, and Baris Kasikci. 2018. Cntr: Lightweight OS Containers. In *2018 USENIX Annual Technical Conference (USENIX ATC 18)*. USENIX Association, Boston, MA, 199–212. https://www.usenix.org/conference/atc18/presentation/thalheim

[50] Shivaram Venkataraman, Zongheng Yang, Michael Franklin, Benjamin Recht, and Ion Stoica. 2016. Ernest: Efficient performance prediction for Large-Scale Advanced Analytics. In *13th USENIX symposium on networked systems design and implementation (NSDI 16)*. 363–378.

[51] Ao Wang, Shuai Chang, Huangshi Tian, Hongqi Wang, Haoran Yang, Huiba Li, Rui Du, and Yue Cheng. 2021. FaaSNet: Scalable and Fast Provisioning of Custom Serverless Container Runtimes at Alibaba Cloud Function Compute. In *2021 USENIX Annual Technical Conference (USENIX ATC 21)*. USENIX Association, 443–457. https://www.usenix.org/conference/atc21/presentation/wang-ao

[52] Bicheng Yang, Yuanpei Wu, Rongchuan Liu, Dong Du, Yubin Xia, Binyu Zang, and Haibo Chen. 2026. Making Hardware Features Bespoke: Operating System Support for Bes-Mode Processes. *ACM Trans. Comput. Syst.* 44, 3, Article 20 (Aug. 2026), 29 pages. doi:10.1145/3802921

[53] Wayne Xin Zhao, Xudong Zhang, Daniel Lemire, Dongdong Shan, Jian-Yun Nie, Hongfei Yan, and Ji-Rong Wen. 2015. A General SIMD-Based Approach to Accelerating Compression Algorithms. *ACM Trans. Inf. Syst.* 33, 3, Article 15 (March 2015), 28 pages. doi:10.1145/2735629